An exploration on Solving Traveling Salesman Problem with Image-based Classification

Mohsen Jafari

Computer science

Bowling Green State University

Bowling Green, Ohio, United States

mjafari@bgsu.edu

Abstract - Solving combinatorial optimization problems needs developing high-quality algorithms. One of the famous problems in this category is the traveling salesman problem (TSP). This article focuses on solving this problem. We are using deep learning algorithms, specifically the Pixel-mapped Classification Network (PCN), to approximate the evaluation of vertices and construct a tour. This approach treats the problem as an image and selects appropriate vertices using classification. We use greedy selection and beam-search algorithms with PCN evaluation and verify their effectiveness in improving solution quality. Using deep learning algorithms for solving combinatorial optimization problems will be discussed deeply in this article.

Index Terms—Combinatorial Optimization Problem, Traveling Salesman Problem, Deep Learning

I - Introduction

The traveling salesman problem (TSP) is a fundamental combinatorial optimization problem in computer science. TSP involves finding a tour that has minimum total edge cost and visits all vertices only once. Exact algorithms and heuristics are two categories of algorithms used for solving optimization problems [1]. Machine learning techniques, particularly deep learning, have shown high performance in many problems, including image processing [4][5]. We work on a method for solving TSP using a convolutional neural network (CNN) that treats TSP as an image and approximates the image of the solution. The Convolutional Neural Network (CNN) takes in an image that displays all the points on a TSP graph and generates an approximate image of the most optimal tour [2][3]. In order to determine the quality of each edge, the 'color depth' of the line connecting each pair of vertices is calculated. Proposing a method to make better use of the approximation ability of CNN by showing the proper element to choose next during solving instead of the whole tour at once. In this method, a solution is constructed by selecting vertices and extending a partial path repeatedly. We are trying to offer a new method to address the limitations of using CNN for solving the Traveling Salesman Problem (TSP). Instead of the CNN outputting the entire tour at once, we suggest using the CNN to select the next vertex to visit during the construction of the tour. The CNN evaluates each vertex and suggests the one with the highest likelihood of being selected next. This process is repeated until a complete tour is constructed [4][6].

The beam search can be mentioned as tree search instead of greedy selecting of the vertex has the max evaluation. However, the computation time increases too much using normal beam search, so we also consider about a method to reduce the computation time by restricting beam branching according to the evaluations [7].

II - Describing used algorithms

A. Convolutional Neural Network

CNNs are a type of deep neural network used for image recognition and processing. They use a specialized mathematical operation called convolution to extract features from images. The convolutional layers in CNNs are made up of filters or kernels that perform the convolution operation. Pooling layers are used to down sample the feature maps generated by the convolutional layers. CNNs can be used for a variety of applications such as image classification, object detection, and facial recognition. Transfer learning is a popular technique used in CNNs to leverage pre-trained models for specific tasks. Different types of activation functions such as “ReLU” and “softmax” are used in CNNs to introduce non-linearity in the network. Regularization techniques such as dropout and weight decay are used to prevent overfitting in CNNs. CNNs can be trained using various optimization algorithms such as stochastic gradient descent (SGD) and Adam. CNNs have been successful in achieving state-of-the-art performance in various image-related tasks and are widely used in industry and academia.[9]

B. Heuristics

The heuristics is a problem-solving techniques that use rules of thumb and educated guesses to arrive at a solution. Heuristics are used in artificial intelligence (AI) and machine learning (ML) to create more efficient and effective algorithms. There are some lists of common heuristics used in AI, such as hill climbing, genetic algorithms, and simulated annealing. The heuristics should be used cautiously, as they can sometimes lead to incorrect conclusions or biased results. The heuristics can be improved through experimentation and feedback, and that they are particularly useful in situations where complete information is unavailable or impractical to obtain. There is an importance of domain knowledge in the development and application of heuristics. The growing importance of heuristics in the field of AI as researchers seek is leaded to create more robust and adaptable algorithms.[10]

C. Beam Search Algorithm

The beam search strikes a balance between the greedy search and the exhaustive search at the expense of introducing one hyper-parameter β, which is a positive integer specifying the number of paths (beams) we keep while running a beam search.

For instance, with β = 2, we only keep the first and the second probable sequence at each step. Let’s go through an example where we have only four items in output vocabulary:

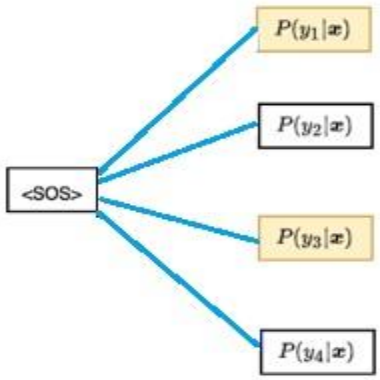

Fig. 1. First Step.

Suppose outputs y1 and y3 are the first and second probable outputs, we continue only with them to the second step:

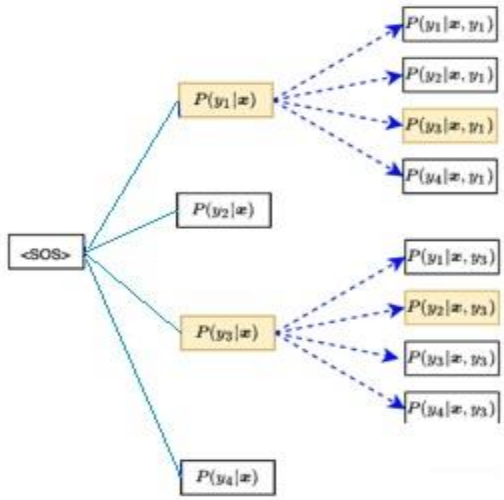

Fig. 2. Second Step.

We use only the top two sequences: (y1, y3) and (y3, y2). So, we always keep the two most probable beams, avoiding the curse of dimensionality problem.
When β=1, the beam search becomes the greedy search. When β is not limited, it becomes an exhaustive search.[11]

III - Problems

A.Traveling Salesman Problem

The Traveling Salesman Problem (TSP) is a problem of finding the shortest tour that visits all vertices only once. It is represented by a graph with vertices and edges, and its solution is a tour that visits all vertices exactly once. The cost of each edge is usually considered as a distance between two locations. If all vertices are on a 2D plane, and the cost is defined as the Euclidean distance, it is called the 2D Euclidean TSP. The TSP is a problem that belongs to NP-hard and has a large number of unique solutions that exponentially increase as the number of vertices increase. As you can see in fig 1 there is an example of the optimal and a solution of TSP.

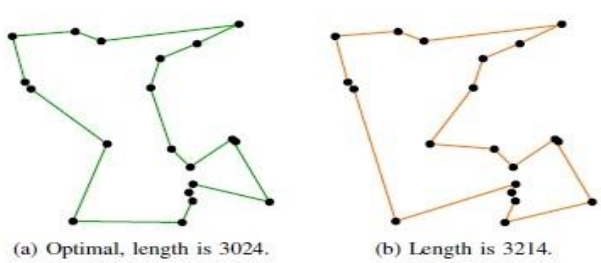

Fig. 3. Example of the optimal and a solution of TSP.

B. Heuristics

We discuss about various heuristics used for the Traveling Salesman Problem (TSP). The simplest method is the Nearest Neighbor algorithm, which constructs a tour by repeatedly moving to the closest vertex. However, it tends to select longer edges towards the end. Another method is the Farthest Insertion algorithm, which expands a sub-tour by inserting the farthest vertex at an appropriate position to make the next sub-tour shortest. The 2-opt search is a local search method that uses the 2-opt operation to reconnect two edges and move to a shorter tour. It guarantees no crossing of edges in the local minimum solution.

IV. MODELS

A. Pixel-mapped Classification Network

The Pixel-mapped Classification Network (PCN) is a proposed method that employs a Convolutional Neural Network (CNN) to tackle the Traveling Salesman Problem (TSP). By treating the TSP as an image, each vertex is evaluated using image layers and the input and output images are represented in a discrete pixel space. The state s is composed of layers that include the starting and ending vertices of the path and the remaining vertices. In each constructive iteration, a set of image layers are fed to the CNN, which outputs likelihoods for selecting the next vertex, and the vertex with the highest evaluation is added to the end of the partial path. The proposed method constructs a tour by repeatedly selecting vertices and adding them to the partial path. Although the experiments in Section IV only use the first three layers, additional elements such as images of all vertices and the partial path could potentially improve the accuracy of the solution. Algorithm 1, Fig. 2 show the procedure of finding a solution that we plan.

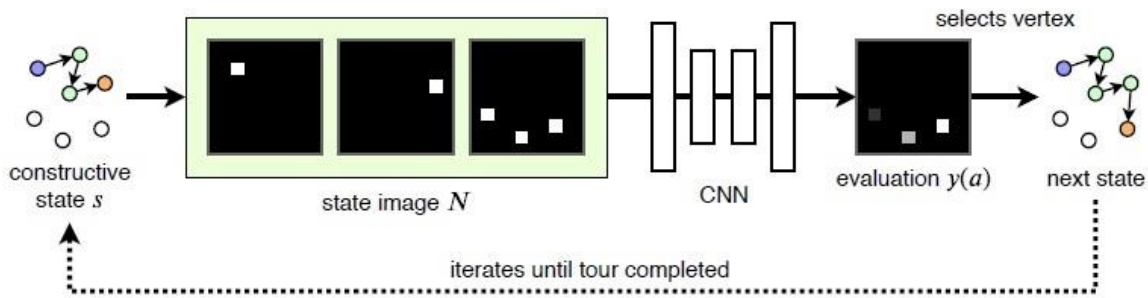

Fig. 4. An iteration of constructing a solution using PCN

B. Training model

A convolutional neural network (CNN) is trained to select the next vertex in a given tour. The task is treated as a classification problem where the output likelihood of a pixel having a vertex is the highest. The loss function used is cross-entropy, which is defined based on a teacher image that provides information about the optimal tour. The teacher image is an one-hot expression that indicates if the next vertex in the optimal tour exists at a particular pixel location. The constructive state fed to the CNN is sampled randomly from the optimal tour. If the optimal tour for the instance used for training is known, the loss function L is defined as cross-entropy as follows:

$$p(a) = \mathrm{softmax}(y(a)) = \frac{e^{y(a)}}{\sum_{h \in A} e^{y(h)}} \quad a \in A \quad (2)$$

$$\mathcal{L} = -\sum_{a \in A} t(a) \ln p(a) \quad (3)$$

C. Restricted Beam Search

The proposed model has a classification ability that can be effectively used to find a higher-quality solution. A tree search using beam search with the outputs of PCN as the evaluation of the vertices is suggested. The PCN outputs higher values at pixels that are more likely to be traced next. Top k vertices with high output are selected as branches for search. However, the number of searches increases exponentially for the number of vertices n, which makes computation time too long in large-scale instances. To reduce search time, the number of searches is limited by selecting the next vertex based on the maximum likelihood obtained from the output of the model. If the maximum likelihood is not less than the threshold β, a vertex with max output is selected deterministically. Bounding is also used to reduce search time.

When selecting the next vertex, the maximum likelihood pmax|R obtained from the output of model as follows:

$$p'(\psi_j) = \mathrm{softmax}(y(\psi_j)) = \frac{e^{y(\psi_j)}}{\sum_{i \in R} e^{y(\psi_i)}} \quad j \in R (4)$$

$$p_{\max|R} = \max\{p'(\psi_j) \mid j \in R\} \quad (5)$$

The cost of the best solution is recorded during the search, and if the total cost of the partial path under construction becomes larger than the recorded cost, the branch is pruned.

V - EXPERIMENTS

The Computer experiments evaluating the performance of a proposed algorithm implemented in the Keras machine learning framework using Python 3.6 and Tensorflow backend. The experiments were conducted on a computer with an Intel Core i7-6900K 3.20GHz CPU, 125.7 GiB memory, and a NVIDIA GeForce GTX TITAN X GPU. The dataset of 2D Euclidean TSP comprised of 182,000 instances with 10-100 vertices for training and validation, and some instances with 20-400 vertices to solve the solution. The TSP solver Concorde was used to find the optimal solution for each instance [8]. The PCN was trained by supervised learning using a CNN consisting of 22 layers of convolution layers and 4 layers of transposed convolution layers. The classification accuracy has reached more than 95% for both training and validation. The performance of the trained model was examined by solving the test instances of TSP and evaluating its quality. The proposed algorithms of PCN and PCN with restricted beam search (PCN-BS) were used with beam width k = 2 and branching threshold β ∈ 0.3, 0.5, 0.6. Comparing methods were some heuristics of nearest neighbor (NN), farthest insertion (FI), and Christofides. The error ratio ε(z) of the cost for the optimal solution was calculated, and the error ratio and computation time for each solution method were shown in Table I, Table II.

(a) Optimal. (b) PCN ($\varepsilon = 0.122\%$). (c) PCN-BS(β=0.5) ($\varepsilon = 0.070\%$).

Fig. 5. Solutions of rand50-A.

(a) Optimal. (b) PCN ($\varepsilon = 1.155\%$). (c) PCN-BS(β=0.5) ($\varepsilon = 0.878\%$).

Fig. 6. Solutions of rand100-A.

The error ratio of the PCN was lower than that of all comparison methods, even when restricted beam search was not used. The error ratio tends to increase as the number of vertices increases, which seems to be because the PCN was trained using instances of a number of vertices of 10-100. In order to adapt and to obtain a better error ratio for larger instances, it is necessary to use them as training data. When beam search is applied, the error ratio is improved as the branching threshold β is increased. However, the time of computing increases over 8,000 seconds at n = 400. Therefore, there is a need for a search method that can find high-quality solutions in a shorter time.

VI. CONCLUSION

We try to use the algorithms using CNN for 2D Euclidean TSP and evaluates their performances through experiments. The algorithms use a Pixel-mapped Classification Network to determine the next vertex based on images and employ beam search to find higher quality solutions. The experiments show that the algorithms outperform conventional methods, and applying beam search with branching further improves the solutions. However, finding optimal solutions for large-scale instances is difficult, and future work suggests using reinforcement learning to train the model without a teacher. It could be better to offer strengthening the CNN output to make a shorter tour.

VII - Suggestion to Author

After I read the paper I think it would be a great work to use this approach for solving other NP-Complete problems such as Hamiltonian Path or 3DM. We can figure out if this solving approach solves other problems or not? As a further work for this particular paper I mention that Is it possible to create 3D Euclidean TSP. In real world we always have 3 dimensions for addressing a place , so extending this problem to 3 dimensions may help solving real world NP-Complete problems.

References

[1] Y. Nagata and S. Kobayashi, "A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem," INFORMS Journal on Computing, vol. 25, no. 2, pp. 346–363, 2013. [Online]. Available: https://doi.org/10.1287/ijoc.1120.0506

[2] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," arXiv preprint arXiv:1511.06434, 2015.

[3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," In CVPR 2017, arXiv:1611.07004, 2016.

[4] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," arXiv preprint arXiv:1506.03134, 2015.

[5] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," arXiv preprint arXiv:1611.09940, 2016.

[6] H. Dai, E. B. Khalil, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," arXiv preprint arXiv:1704.01665, 2017.

[7] S. Miki, D. Yamamoto, and H. Ebara, "Applying deep learning and reinforcement learning to traveling salesman problem," in 2018 International Conference on Computing, Electronics Communications Engineering (iCCECE), 2018.

[8] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, "Concorde tsp solver," http://www.math.uwaterloo.ca/tsp/concorde/, 2006, accessed 2018/2/22.

[9] Mandal, M. (2021, May 1). CNN for deep learning. Analytics Vidhya.

https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/

[10] What is Heuristics? Definition, Working, and Examples. (n.d.). Spiceworks. Retrieved April 19, 2023, from

https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-heuristics/

[11] Beam search for machine translation: How greedy, exhaustive, and beam search algorithms work - KiKaBeN. (2021, October 17). KiKaBeN - Smart Tech Information: From Concept to Coding. https://kikaben.com/beam-search-for-machine-translation/

TABLE I
AVERAGE ERROR RATIO (%).

Instances	**PCN**	**PCN-BS(β=0.3)**	**PCN-BS(β=0.5)**	**PCN-BS(β=0.6)**	NN	NN+2opt	FI+2opt	christofides+2opt
rand20-A	0.196	0.515	**0.047**	**0.047**	20.587	2.099	1.263	4.498
rand20-B	**0.000**	0.455	**0.000**	**0.000**	10.778	2.826	2.407	2.515
rand50-A	0.240	0.334	**0.094**	**0.094**	22.104	4.718	4.690	0.703
rand50-B	0.283	0.283	0.364	**0.243**	16.761	6.113	4.089	6.397
rand75-A	1.325	1.626	0.882	**0.375**	20.933	4.007	5.718	3.052
rand75-B	0.990	0.740	0.774	**0.175**	13.747	4.257	6.356	6.062
rand90-A	1.517	1.142	0.809	**0.440**	17.437	4.153	5.409	2.872
rand90-B	1.921	2.242	0.836	**0.279**	23.704	5.610	9.388	4.052
rand100-A	1.344	1.095	0.961	**0.540**	30.905	5.219	6.296	4.776
rand100-B	1.079	1.147	0.369	**0.237**	31.031	8.532	6.378	4.457
rand100-C	0.851	1.400	1.041	**0.566**	18.249	4.568	6.674	3.211
rand150-A	1.796	1.945	1.499	**0.726**	24.039	7.190	6.564	4.017
rand200-A	2.551	2.544	2.356	**2.015**	23.444	6.068	8.099	6.074
rand300-A	4.031	3.927	**2.904**	3.054	22.234	6.586	7.111	4.487
rand400-A	5.379	**5.200**	5.228	5.896	27.351	8.057	8.789	7.289
Total Avg.	1.567	1.640	1.211	**0.979**	21.554	5.334	5.949	4.298

TABLE II
AVERAGE OF COMPUTATION TIME (s) AND TIMEOUT RATIO.

Instances	**PCN**	**PCN-BS(β=0.3)**	**PCN-BS(β=0.5)**	**PCN-BS(β=0.6)**	NN	NN+2opt	FI+2opt	christofides+2opt
rand20-A	0.57	0.57	0.95	1.27	0.00	0.00	0.00	0.00
rand20-B	0.57	0.58	0.90	1.29	0.00	0.00	0.00	0.01
rand50-A	1.53	1.54	2.49	6.79	0.00	0.02	0.01	0.09
rand50-B	1.53	1.54	2.39	6.40	0.00	0.01	0.01	0.04
rand75-A	2.33	2.34	5.17	25.20	0.00	0.05	0.02	0.26
rand75-B	2.33	2.34	6.52	36.47	0.00	0.05	0.02	0.17
rand90-A	2.80	2.81	24.94	209.05	0.00	0.09	0.02	0.41
rand90-B	2.82	2.83	7.70	66.74	0.00	0.10	0.02	0.29
rand100-A	3.11	3.42	16.21	220.22	0.00	0.18	0.03	0.31
rand100-B	3.11	3.13	10.14	121.62	0.00	0.17	0.03	0.49
rand100-C	3.12	3.40	16.94	139.50	0.00	0.11	0.03	0.39
rand150-A	4.74	5.20	53.47	2577.39 (80%)	0.00	0.41	0.10	1.28
rand200-A	6.35	6.95	575.13	4000.02 (100%)	0.00	1.10	0.13	3.33
rand300-A	9.44	11.67	6000.02 (100%)	6000.02 (100%)	0.00	2.83	0.57	6.19
rand400-A	12.57	53.40	8000.01 (100%)	8000.01 (100%)	0.01	7.64	1.14	17.81